RATAN TATA MAHARASHTRA STATE SKILLS UNIVERSITY

School of Science Engineering and Technology Skills

Department of Engineering

Subject – DevOps and Automation Lab

Lab Assignment 3

Aim:

To create a local Git repository and understand Git branching by performing basic Git operations such as add, commit, status, history, and branch management.

Learning Objectives:

After completing this lab, the student will be able to:

1. Understand the concept of Git branching.
2. Create and initialize a local Git repository.
3. Track file changes using Git commands.
4. Create and switch between branches.
5. Maintain commit history using Git.
6. Differentiate between staging and committing changes.

Theory:

1. What is a Master Branch?

The master branch functions as the central and default development path in a Git repository. It holds the project's stable, production-grade codebase. Typically, developers build specific features on isolated branches before merging them back into the master branch for the final release.

2. Difference Between git add and git commit.

- **git add:** Moves changes from the working directory to the **Staging Area** (Index). Used to select specific modifications for the next snapshot.
- **git commit:** Records staged changes permanently in the **Local Repository** history.
- **Shortcut:** *git commit -a* stages and commits all tracked modified files simultaneously. However, *git add* is mandatory for new (untracked) files.

3. Basic Git Commands & Syntax:

Command	Syntax	Description
git init	git init	Initializes a new local Git repository.
git status	git status	Displays the state of the working directory and staging area.
git add	git add <file>	Adds specific files (or . for all) to the staging area.
git commit	git commit -m "message"	Saves the staged snapshot to the project history with a message description about commit.
git branch	git branch <name>	Creates a new branch at the current commit.
git checkout	git checkout <name>	Switches the current HEAD to the specified branch.
git log	git log	Displays a chronological list of commits.

Execution Procedure:

Note: This procedure presumes git config (user name and email) has already been set up.

- **Step 1:** Create a new directory named git-lab.
  - mkdir git-lab
- **Step 2:** Navigate into the newly created directory.
  - cd git-lab
- **Step 3:** Initialize the local Git repository.
  - git init
- **Step 4:** Check the initial status of the repository.
  - git status
- **Step 5:** Create a new file named test.txt.
  - touch test.txt
- **Step 6:** Add the created file to the staging area.
  - git add test.txt
- **Step 7:** Commit the staged file to the master branch.
  - git commit -m "First commit"
- **Step 8:** Create a new development branch named dev.
  - git branch dev
- **Step 9:** Switch from the master branch to the dev branch.
  - git checkout dev
- **Step 10:** Modify the contents of the file while inside the dev branch.
  - echo "Branch work" > test.txt
- **Step 11:** Add the modified file to the staging area.
  - git add test.txt
- **Step 12:** Commit the changes made in the dev branch.
  - git commit -m "Second commit in dev branch"
- **Step 13:** Display the full commit history to verify both commits.
  - git log

Screenshot(s):

```
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps
$ mkdir git-lab

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps
$ cd git-lab/

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab
$ git init
Initialized empty Git repository in C:/Users/shrey/Documents/MSSU/MSSU_Sem_6/Dev
Ops/git-lab/.git/

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (master)
$ touch test.txt
g
shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (master)
$ git add test.txt

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (master)
$ git commit -m "First Commit"
[master (root-commit) 7f6cc64] First Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test.txt

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (master)
$ git branch dev

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (master)
$ git checkout dev
Switched to branch 'dev'

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (dev)
$ echo "Branch work" > test.txt

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (dev)
$ git add test.txt

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (dev)
$ git commit -m "Second commit in dev branch"
[dev d8c7278] Second commit in dev branch
 1 file changed, 1 insertion(+)

shrey@Shreya MINGW64 ~/Documents/MSSU/MSSU_Sem_6/DevOps/git-lab (dev)
$ git log
commit d8c7278641ec421c2bb1213c08d51e119e844abb (HEAD -> dev)
Author: Sonnenglanz <shreyadpazare@gmail.com>
Date:   Fri Feb 6 17:00:20 2026 +0530

    Second commit in dev branch

commit 7f6cc64c3f4e045cabb898ca6873c92b2b65b7f5 (master)
Author: Sonnenglanz <shreyadpazare@gmail.com>
Date:   Fri Feb 6 16:58:39 2026 +0530

    First Commit
```

Result:

Successfully created a Git repository and performed branching operations using Git commands including init, add, commit, branch, and checkout.

Conclusion:

This experiment demonstrates the use of Git branching and basic Git commands. Understanding the flow from the working directory to the staging area and finally the repository is essential for version control in DevOps and automation workflows.